

Punto 3.8 - Eager Loading con Include y ThenInclude

Práctica

Ejercicio: Añadir métodos de carga Eager al repositorio de órdenes del proyecto AceriaData. Crear métodos que usen Include, ThenInclude, Filtered Include, AsSplitQuery y AutoInclude. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

Contexto del proyecto: En el punto 3.7 se añadieron consultas con joins y navegación, incluyendo Join, SelectMany, DefaultIfEmpty y GroupJoin. En este punto se profundiza en la carga Eager, incluyendo Include, ThenInclude, Filtered Include y AsSplitQuery. Estas técnicas se usarán en el punto 3.9 para la carga Lazy y en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de carga Eager a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleInclude();
    List<OrdenFabricacion> ObtenerOrdenesConAleacionesInclude();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivas();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleSplitQuery();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerOrdenesConPlanchasInclude(); → método que carga las órdenes con planchas usando Include.

Línea 11: List ObtenerOrdenesConPlanchasYDetalleInclude(); → método que carga las órdenes con planchas y detalle usando Include.

Línea 12: List ObtenerOrdenesConAleacionesInclude(); → método que carga las órdenes con aleaciones usando Include y ThenInclude.

Línea 13: `List ObtenerOrdenesConPlanchasActivas();` → método que carga las órdenes con planchas activas usando `Filtered Include`.

Línea 14: `List ObtenerOrdenesConPlanchasYDetalleSplitQuery();` → método que carga las órdenes con planchas y detalle usando `AsSplitQuery`.

Línea 16: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 17: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o DTOs concretos.

| Paso 3: Implementar los métodos de carga Eager en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConAleacionesInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivas()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas.Where(p => p.Activa))
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasYDetalleSplitQuery()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: `public List ObtenerOrdenesConPlanchasInclude();` → declara el método que carga las órdenes con planchas.

Línea 3: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 5: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 6: `.ToList();` → materializa la consulta.

Línea 9: `public List ObtenerOrdenesConPlanchasYDetalleInclude()` → declara el método que carga las órdenes con planchas y detalle.

Línea 11: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 12: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 13: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 14: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 15: `.ToList();` → materializa la consulta.

Línea 18: `public List ObtenerOrdenesConAleacionesInclude()` → declara el método que carga las órdenes con aleaciones.

Línea 20: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 21: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 22: `.ThenInclude(oa => oa.Aleacion)` → incluye la aleación de cada entidad intermedia.

Línea 23: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 24: `.ToList();` → materializa la consulta.

Línea 27: `public List ObtenerOrdenesConPlanchasActivas()` → declara el método que carga las órdenes con planchas activas.

Línea 29: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 30: `.Include(o => o.Planchas.Where(p => p.Activa))` → incluye solo las planchas activas.

Línea 31: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 32: `.ToList();` → materializa la consulta.

Línea 35: `public List ObtenerOrdenesConPlanchasYDetalleSplitQuery()` → declara el método que carga las órdenes con planchas y detalle usando `AsSplitQuery`.

Línea 37: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 38: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 39: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 40: `.AsSplitQuery()` → divide la consulta en varias.

Línea 41: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 42: `.ToList();` → materializa la consulta.

Error común: si se incluyen varias colecciones sin `AsSplitQuery`, EF Core genera un producto cartesiano. Se debe usar `AsSplitQuery` para evitar el problema.

| Paso 4: Crear el caso de uso de carga Eager

Crear el archivo `src/AceriaData.Application/UseCases/CargaEagerUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class CargaEagerUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public CargaEagerUseCase(IUnidadDeTrabajo unidad)
```

```

{
    _unidad = unidad;
}

public void Ejecutar()
{
    Console.WriteLine("=== CARGA EAGER CON INCLUDE ===");

    DemostrarIncludePlanchas();
    DemostrarIncludePlanchasYDetalle();
    DemostrarThenIncludeAleaciones();
    DemostrarFilteredInclude();
    DemostrarSplitQuery();
    DemostrarSqlInclude();
}

private void DemostrarIncludePlanchas()
{
    Console.WriteLine("\n--- Include: órdenes con planchas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | Planchas:
{orden.Planchas.Count}");
    }
}

private void DemostrarIncludePlanchasYDetalle()
{
    Console.WriteLine("\n--- Include: órdenes con planchas y detalle ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude();
    foreach (var orden in ordenes)
    {
        var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
        Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle:
{detalle}");
    }
}

private void DemostrarThenIncludeAleaciones()
{
    Console.WriteLine("\n--- ThenInclude: órdenes con aleaciones ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConAleacionesInclude();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | Aleaciones: {orden.OrdenesAleaciones.Count}");
        foreach (var ordenAleacion in orden.OrdenesAleaciones)
        {
            Console.WriteLine($" {ordenAleacion.Aleacion.Nombre} | Cantidad:
{ordenAleacion.CantidadUtilizada} kg");
        }
    }
}

private void DemostrarFilteredInclude()
{
    Console.WriteLine("\n--- Filtered Include: órdenes con planchas activas ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasActivas();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count}");
    }
}

private void DemostrarSplitQuery()
{
    Console.WriteLine("\n--- AsSplitQuery: órdenes con planchas y detalle ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery();

```

```

        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");
        }
    }

    private void DemostrarSqlInclude()
    {
        Console.WriteLine("\n--- SQL generado por un Include ---");

        var consulta = _unidad.Ordenes.ObtenerQueryable()
            .Include(o => o.Planchas)
            .Include(o => o.Detalle);

        var sql = consulta.ToQueryString();
        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class CargaEagerUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public CargaEagerUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== CARGA EAGER CON INCLUDE ==="); → muestra la cabecera.

Línea 18: DemostrarIncludePlanchas(); → llama al método de Include con planchas.

Línea 19: DemostrarIncludePlanchasYDetalle(); → llama al método de Include con planchas y detalle.

Línea 20: DemostrarThenIncludeAleaciones(); → llama al método de ThenInclude con aleaciones.

Línea 21: DemostrarFilteredInclude(); → llama al método de Filtered Include.

Línea 22: DemostrarSplitQuery(); → llama al método de AsSplitQuery.

Línea 23: DemostrarSqlInclude(); → llama al método de SQL de Include.

Línea 26: private void DemostrarIncludePlanchas() → declara el método.

Línea 28: Console.WriteLine("\n--- Include: órdenes con planchas ---"); → muestra la cabecera.

Línea 30: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude(); → llama al método del repositorio.

Línea 31: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 33: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | Planchas: {orden.Planchas.Count}"); → muestra los datos.

Línea 36: private void DemostrarIncludePlanchasYDetalle() → declara el método.

Línea 38: Console.WriteLine("\n--- Include: órdenes con planchas y detalle ---"); → muestra la cabecera.

Línea 40: var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude(); → llama al método del repositorio.

Línea 41: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 43: var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica; → comprueba si el detalle existe.

Línea 44: `Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle: {detalle}");` → muestra los datos.

Línea 48: `private void DemostrarThenIncludeAleaciones()` → declara el método.

Línea 50: `Console.WriteLine("\n--- ThenInclude: órdenes con aleaciones ---");` → muestra la cabecera.

Línea 52: `var ordenes = _unidad Ordenes.ObtenerOrdenesConAleacionesInclude();` → llama al método del repositorio.

Línea 53: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 55: `Console.WriteLine($" {orden.NumeroOrden} | Aleaciones: {orden.OrdenesAleaciones.Count}");` → muestra el número de aleaciones.

Línea 56: `foreach (var ordenAleacion in orden.OrdenesAleaciones)` → itera sobre las entidades intermedias.

Línea 58: `Console.WriteLine($" {ordenAleacion.Aleacion.Nombre} | Cantidad: {ordenAleacion.CantidadUtilizada} kg");` → muestra los datos.

Línea 62: `private void DemostrarFilteredInclude()` → declara el método.

Línea 64: `Console.WriteLine("\n--- Filtered Include: órdenes con planchas activas ---");` → muestra la cabecera.

Línea 66: `var ordenes = _unidad Ordenes.ObtenerOrdenesConPlanchasActivas();` → llama al método del repositorio.

Línea 67: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 69: `Console.WriteLine($" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count}");` → muestra los datos.

Línea 73: `private void DemostrarSplitQuery()` → declara el método.

Línea 75: `Console.WriteLine("\n--- AsSplitQuery: órdenes con planchas y detalle ---");` → muestra la cabecera.

Línea 77: `var ordenes = _unidad Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery();` → llama al método del repositorio.

Línea 78: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 80: `Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count}");` → muestra los datos.

Línea 84: `private void DemostrarSqlInclude()` → declara el método.

Línea 86: `Console.WriteLine("\n--- SQL generado por un Include ---");` → muestra la cabecera.

Línea 88: `var consulta = _unidad Ordenes.ObtenerQueryable()` → inicia la consulta.

Línea 89: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 90: `.Include(o => o.Detalle);` → incluye la referencia al detalle.

Línea 92: `var sql = consulta.ToQueryString();` → obtiene el SQL.

Línea 93: `Console.WriteLine(sql);` → imprime el SQL.

Error común: si el método `ObtenerQueryable` no está implementado en el repositorio, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<CargaEagerUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<CargaEagerUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con planchas, detalle y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas, detalle y aleaciones:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del
    Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur",
    Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000,
    Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500,
    Peso = 280.8m, Activa = false };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000,
    Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
    TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    var ordenAleacion1 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" };
    var ordenAleacion2 = new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion2.Id,
    CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" };
    var ordenAleacion3 = new OrdenAleacion { OrdenFabricacionId = orden2.Id, AleacionId = aleacion1.Id,
    CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" };
    context.OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);

    context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var aleacion1 = new Aleacion { ... }; → crea la primera aleación.

Línea 8: var aleacion2 = new Aleacion { ... }; → crea la segunda aleación.

Línea 9: context.Aleaciones.AddRange(aleacion1, aleacion2); → registra las aleaciones.

Línea 10: context.SaveChanges(); → inserta las aleaciones.

Línea 12: var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.

Línea 13: var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.

Línea 14: context.OrdenesFabricacion.AddRange(orden1, orden2); → registra las órdenes.

Línea 15: context.SaveChanges(); → inserta las órdenes.

Línea 17: var plancha1 = new PlanchaAcero { ... }; → crea la primera plancha activa.

Línea 18: var plancha2 = new PlanchaAcero { ... }; → crea la segunda plancha inactiva.

Línea 19: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha activa.

Línea 20: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 22: `var detalle1 = new DetalleOrden { ... };` → crea el detalle.

Línea 23: `context.DetallesOrden.Add(detalle1);` → registra el detalle.

Línea 25: `var ordenAleacion1 = new OrdenAleacion { ... };` → crea la primera relación.

Línea 26: `var ordenAleacion2 = new OrdenAleacion { ... };` → crea la segunda relación.

Línea 27: `var ordenAleacion3 = new OrdenAleacion { ... };` → crea la tercera relación.

Línea 28: `context OrdenesAleaciones.AddRange(ordenAleacion1, ordenAleacion2, ordenAleacion3);` → registra las relaciones.

Línea 30: `context.SaveChanges();` → inserta las planchas, el detalle y las relaciones.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada consulta. Se observan las órdenes con planchas, con planchas y detalle, con aleaciones, con planchas activas, con `AsSplitQuery` y el SQL generado por `Include`.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CARGA EAGER CON INCLUDE ===
```

```
--- Include: órdenes con planchas ---
```

```
OF-2024-0001 | Constructora del Norte | Planchas: 2
OF-2024-0002 | Constructora del Sur | Planchas: 1
```

```
--- Include: órdenes con planchas y detalle ---
```

```
OF-2024-0001 | Planchas: 2 | Detalle: C: 0.45%, Mn: 0.75%
OF-2024-0002 | Planchas: 1 | Detalle: Sin detalle
```

```
--- ThenInclude: órdenes con aleaciones ---
```

```
OF-2024-0001 | Aleaciones: 2
  AISI 1045 | Cantidad: 1500.5 kg
  AISI 4140 | Cantidad: 800.0 kg
OF-2024-0002 | Aleaciones: 1
  AISI 1045 | Cantidad: 1200.0 kg
```

```
--- Filtered Include: órdenes con planchas activas ---
```

```
OF-2024-0001 | Planchas activas: 1
OF-2024-0002 | Planchas activas: 1
```

```
--- AsSplitQuery: órdenes con planchas y detalle ---
```

```
OF-2024-0001 | Planchas: 2
OF-2024-0002 | Planchas: 1
```

```
--- SQL generado por un Include ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ..., [p].[Id], [p].[Espesor], ..., [d].[Id], [d].
[ComposicionQuimica], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [DetallesOrden] AS [d] ON [o].[Id] = [d].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

La primera sección muestra las órdenes con planchas. La segunda sección muestra las órdenes con planchas y detalle. La tercera sección muestra las órdenes con aleaciones. La cuarta sección muestra las órdenes con planchas activas. La quinta sección muestra las órdenes con AsSplitQuery. La sexta sección muestra el SQL generado por Include.

Observaciones: el SQL generado incluye dos LEFT JOIN con las tablas de planchas y detalles. El filtro global de Soft Delete se aplica automáticamente. El Filtered Include filtra las planchas inactivas. AsSplitQuery divide la consulta en varias.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerOrdenesConPlanchasInclude para llamar a ToList antes del Include:

```
csharp
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .ToList()
        .Select(o =>
        {
            o.Planchas = _context.PlanchasAcero.Where(p => p.OrdenId == o.Id).ToList();
            return o;
        })
        .ToList();
}
```

Resultado esperado: el código ejecuta una consulta para cargar todas las órdenes y después una consulta por cada orden para cargar sus planchas. Esto es el problema N+1: una consulta para las órdenes y N consultas para las planchas. Si hay cien órdenes, se ejecutan ciento una consultas.

Solución: usar Include para cargar las planchas en una sola consulta.

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado con la solución: el código ejecuta una sola consulta con un LEFT JOIN para cargar las órdenes y sus planchas. No hay problema N+1.

Errores comunes del ejercicio

Error	Causa	Solución
Producto cartesiano N+1	Se llamó a ToList antes del Include	Se incluyeron varias colecciones sin AsSplitQuery Usar AsSplitQuery
ThenInclude sin Include	Se usó ThenInclude sin Include previo	Aplicar Include antes de ToList Usar Include antes de ThenInclude
NullReferenceException en referencia	La propiedad de navegación es null	Comprobar si la propiedad es null
Filtro no aplicado	Se aplicó el filtro fuera del Include	Aplicar el filtro dentro del Include
AutoInclude excesivo	Se configuró en muchas propiedades	Usar con moderación

Reto resuelto: Consulta con Include, ThenInclude y Filtered Include
 Reto: Crear un método en el repositorio que cargue las órdenes con sus planchas activas y sus aleaciones, usando Include con filtro y ThenInclude. Añadir el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
csharp
List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivasYAleaciones();
```

Línea 1: List ObtenerOrdenesConPlanchasActivasYAleaciones(); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasActivasYAleaciones()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas.Where(p => p.Activa))
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerOrdenesConPlanchasActivasYAleaciones() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas.Where(p => p.Activa)) → incluye solo las planchas activas.

Línea 5: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 6: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 7: .AsSplitQuery() → divide la consulta en varias para evitar el producto cartesiano.

Línea 8: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 9: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarPlanchasActivasYAleaciones()
{
    Console.WriteLine("\n--- Planchas activas y aleaciones ---");

    var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasActivasYAleaciones();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count} |
Aleaciones: {orden.OrdenesAleaciones.Count}");
    }
}
```

Línea 1: `private void DemostrarPlanchasActivasYAleaciones()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Planchas activas y aleaciones ---");` → muestra la cabecera.

Línea 5: `var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasActivasYAleaciones();` → llama al método del repositorio.

Línea 6: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 8: `Console.WriteLine($" {orden.NumeroOrden} | Planchas activas: {orden.Planchas.Count} | Aleaciones: {orden.OrdenesAleaciones.Count}");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarPlanchasActivasYAleaciones();
Paso 5: Ejecutar dotnet run y verificar que las órdenes se muestran con sus planchas activas y sus aleaciones.
```

Resultado esperado: las órdenes se muestran con su número, número de planchas activas y número de aleaciones. `AsSplitQuery` divide la consulta en varias para evitar el producto cartesiano.

Analogía final

La carga Eager en una acéría es como pedirle al archivo central que, además de la carpeta de la orden, adjunte también las carpetas de las planchas y del detalle. En lugar de pedir la carpeta de la orden y después ir a buscar las planchas y el detalle por separado, se pide todo junto. El archivo central prepara un paquete con la orden y sus documentos relacionados. `Include` es como pedir que se adjunte una carpeta. `ThenInclude` es como pedir que se adjunten los documentos de la carpeta adjunta. `Filtered Include` es como pedir que solo se adjunten las planchas activas. `AsSplitQuery` es como pedir que el paquete se prepare en varios envíos separados para evitar que el paquete sea demasiado grande. `AutoInclude` es como pedir que siempre se adjunten las planchas, sin tener que pedirlo cada vez. Así funciona la carga Eager en EF Core: se cargan las entidades relacionadas en una sola consulta, se evita el problema N+1 y se controla qué se carga y qué no.

Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de carga Eager a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `CargaEagerUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas activas e inactivas, detalle y aleaciones.

Ejecutado las demostraciones de `Include`, `ThenInclude`, `Filtered Include` y `AsSplitQuery`.

Analizado el SQL generado por `Include`.

Diagnosticado el error de aplicar ToList antes del Include.

Creado el método ObtenerOrdenesConPlanchasActivasYAleaciones.

| Conexión con el siguiente punto

En este punto se ha profundizado en la carga Eager, incluyendo Include, ThenInclude, Filtered Include, AsSplitQuery y AutoInclude. Se ha analizado el SQL generado y se ha comprobado que EF Core genera LEFT JOIN para las entidades relacionadas. En el siguiente punto se estudiará la carga Lazy, incluyendo su configuración, su funcionamiento y sus riesgos.
